

Hive BT – Code Snippets & Nav2 Options

CMake • BT Nodes • XML Trees • Runner Mapping • API • Hive • Nav2 Integration

CMake Essentials (bt_runner)

CMakeLists.txt (bt_runner)

```
cmake_minimum_required(VERSION 3.5)
project(bt_runner)

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_action REQUIRED)
find_package(ament_index_cpp REQUIRED)
find_package(behaviortree_cpp REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(hive_interfaces REQUIRED)
find_package(bt_turtle_demo REQUIRED)

add_executable(bt_runner_node src/runner.cpp)
ament_target_dependencies(bt_runner_node
  rclcpp rclcpp_action ament_index_cpp behaviortree_cpp
  sensor_msgs geometry_msgs hive_interfaces bt_turtle_demo)

target_include_directories(bt_runner_node PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>)

install(TARGETS bt_runner_node DESTINATION lib/${PROJECT_NAME})
install(DIRECTORY include/ DESTINATION include)
ament_export_include_directories(include)
install(DIRECTORY trees/ DESTINATION share/${PROJECT_NAME}/trees)

# ament index marker (discoverability)
file(WRITE ${CMAKE_CURRENT_BINARY_DIR}/${PROJECT_NAME} "")
install(FILES ${CMAKE_CURRENT_BINARY_DIR}/${PROJECT_NAME}
  DESTINATION share/ament_index/resource_index/packages)
install(FILES package.xml DESTINATION share/${PROJECT_NAME})

ament_package()
```

GoToA Leaves (nodes_goto.hpp) – key parts

PublishGoalPose snippet

```
// PublishGoalPose: publish PoseStamped to /goal_pose (defaults if no ports)
static BT::PortsList providedPorts() {
    return { BT::InputPort<double>("x"),
             BT::InputPort<double>("y"),
             BT::InputPort<double>("yaw_deg"),
             BT::InputPort<std::string>("frame_id") };
}

BT::NodeStatus tick() override {
    // defaults
    double x=-1.0, y=0.46, yaw_deg=0.0; std::string frame="map";
    (void)getInput("x", x); (void)getInput("y", y);
    (void)getInput("yaw_deg", yaw_deg); (void)getInput("frame_id", frame);
    // build PoseStamped and publish to /goal_pose ...
    return BT::NodeStatus::SUCCESS;
}
```

WaitAtTarget (distance to target via /amcl_pose)

WaitAtTarget snippet

```
static BT::PortsList providedPorts() {
    return { BT::InputPort<double>("x"),
             BT::InputPort<double>("y"),
             BT::InputPort<double>("tol_m"),
             BT::InputPort<int64_t>("timeout_ms") };
}

BT::NodeStatus onRunning() override {
    if (!has_pose_) return BT::NodeStatus::RUNNING;
    double dx = last_x_ - target_x_;
    double dy = last_y_ - target_y_;
    double dist = std::sqrt(dx*dx + dy*dy);
    if (dist <= tol_m_) return BT::NodeStatus::SUCCESS;
    if (elapsed_ms() >= timeout_ms_) return BT::NodeStatus::FAILURE;
    return BT::NodeStatus::RUNNING;
}
```

BT XML and Runner Mapping

bt_runner/trees/go_to_a.xml

```
<root BTCPP_format="4">
  <BehaviorTree ID="GoToA">
    <Sequence>
      <PublishGoalPose x="-1.0" y="0.46" yaw_deg="0.0" frame_id="map"/>
      <WaitAtTarget x="-1.0" y="0.46" tol_m="0.2" timeout_ms="60000"/>
    </Sequence>
  </BehaviorTree>
</root>
```

runner.cpp mapping

```
// inside execute()
std::string behavior = goal->behavior_name;
if (behavior.empty()) {
  if (goal->id == 1) behavior = "SimpleTurtle";
  else if (goal->id == 2) behavior = "GoToA";
  else { /* abort unknown */ }
}

// pick XML by name
std::string xml_path;
if (behavior == "SimpleTurtle") {
  xml_path = ament_index_cpp::get_package_share_directory("bt_turtle_demo") +
    "/trees/turtle_tree_simple.xml";
} else if (behavior == "GoToA") {
  xml_path = ament_index_cpp::get_package_share_directory("bt_runner") +
    "/trees/go_to_a.xml";
}
```

Hive Server – Async forwarding & Status

server.cpp (core fragments)

```
// feedback callback relays to API client + /hive/status
opts.feedback_callback = [this, sh](auto, const auto fb) {
    if (!sh->is_executing()) return;
    publish_status(sh->get_goal()->task_id, "FEEDBACK",
                  "state=" + fb->current_state +
                  ", progress=" + std::to_string(fb->progress_percent));
    auto relay = std::make_shared<ExecuteBehavior::Feedback>(*fb);
    sh->publish_feedback(relay);
};

// result polling (no second executor)
auto result_future = client_->async_get_result(runner_handle);
while (rclcpp::ok()) {
    if (result_future.wait_for(200ms) == std::future_status::ready) {
        auto wrapped = result_future.get();
        auto res = wrapped.result;
        if (wrapped.code == rclcpp_action::ResultCode::SUCCEEDED && res && res->success)
            publish_status(in->task_id, "RESULT", "SUCCESS log=" + res->log_file);
            sh->succeed(res);
        } else {
            publish_status(in->task_id, "RESULT", "FAILURE outcome=" + res->outcome_text);
            sh->abort(res);
        }
        return;
    }
    std::this_thread::sleep_for(50ms);
}
```

API – FastAPI /tasks endpoint (Python)

```
hive_api_gateway/main.py (concept)
app = FastAPI()

@app.post('/tasks')
def create_task(req: TaskRequest):
    task_id = str(uuid.uuid4())
    # build ExecuteBehavior.Goal
    goal = ExecuteBehavior.Goal()
    goal.id = req.id
    goal.behavior_name = req.behavior_name or ""
    goal.task_id = task_id
    # send to hive via rclpy action client (async)
    # return 202 Accepted with task_id
    return {"accepted": True, "task_id": task_id}
```

Nav2 Integration Options for GoToA

Option A – Keep it simple (current) ■ • Publish PoseStamped to /goal_pose. ■ • WaitAtTarget listens to /amcl_pose

Option B – Use NavigateToPose action directly (BehaviorTree leaf) ■ • Create a BT leaf that is an action client to

Option C – Use /tf (odom) instead of /amcl_pose ■ • If AMCL is unavailable, compute robot→target distance using

Testing Checklist

- Build order: hive_interfaces → bt_turtle_demo → bt_runner → hive_bt_server →
- Source overlay: source install/setup.bash (new shell)
- Start: runner, hive, api (3 terminals)
- Trigger: curl -X POST http://localhost:8080/tasks -H 'Content-Type: application/'
- Groot2: connect to localhost:1667; slow tick_ms for clarity
- Observe /hive/status (JSON-ish): ROUTED, FEEDBACK, RESULT
- Verify logs: task_<task_id>.btlog
- Safety stop: ensure runner publishes zero Twist at terminal states